PDF Malware Analysis Toolkit

Full Project Submission Report

Submitted by: Amar Kant Yadav

---

1. Project Overview

This project focuses on designing and developing a practical PDF Malware Analysis Toolkit capable of performing static analysis on suspicious PDF documents. PDF files are frequently abused in phishing campaigns, exploit delivery, embedded malware payloads, and social engineering attacks.

The objective of this project is to simulate real-world Security Operations Center (SOC) analysis workflows by inspecting malicious PDF artifacts, extracting embedded components, identifying Indicators of Compromise (IOCs), and generating structured professional reports.

The toolkit provides automated detection capabilities while maintaining analyst-level interpretability.

---

2. Practical Motivation

PDF documents are widely trusted and commonly exchanged through email and web platforms. Threat actors exploit this trust to:

- Deliver phishing lures
- Trigger embedded JavaScript exploits
- Deploy malware payloads
- Exploit vulnerable PDF reader software
- Execute obfuscated code upon document open

This project provides hands-on experience in malware triage and artifact inspection used by:

- SOC Analysts
- Incident Responders
- Threat Hunters
- Digital Forensic Analysts

---

3. Project Objectives

The primary objectives of this project are:

- Understand how malicious PDFs are structured
- Detect embedded JavaScript and exploit indicators
- Extract suspicious objects and streams
- Identify obfuscation techniques
- Extract network-based IOCs (URLs and IPs)
- Implement automated risk scoring
- Generate a professional malware analysis report

---

4. Tools and Technologies Used

Programming Language

- Python 3.9

Libraries Used

- PyPDF2 (metadata extraction)
- reportlab (professional PDF report generation)
- hashlib (hash calculation)
- re (regex-based detection)

External Malware Analysis Tools (Referenced for Testing)

- pdfid (keyword detection)
- pdf-parser.py (deep object parsing)
- peepdf (interactive PDF analysis)
- qpdf (stream decompression)
- strings (content inspection)

---

5. Methodology and Workflow

The toolkit follows a structured static malware analysis workflow.

Step 1: Load PDF File

- Accept PDF input from command line
- Calculate SHA256 file hash
- Initialize analysis modules

Step 2: Metadata Extraction

- Extract Author
- Extract Creation Date
- Extract Modification Date
- Detect anomalies such as identical timestamps or missing fields

Step 3: Keyword-Based Detection

Scan for suspicious PDF keywords:

- /JavaScript
- /JS
- /OpenAction
- /Launch
- /EmbeddedFile
- /AA

Presence of these keywords indicates potential malicious behavior.

Step 4: Object Enumeration

- Count total objects
- Count total streams
- Identify embedded content structures

Step 5: JavaScript Analysis

- Detect JavaScript presence
- Identify suspicious functions:
    - eval()
    - unescape()
    - atob()
    - fromCharCode()
- Detect potential obfuscation patterns

Step 6: IOC Extraction

Using regular expressions, the toolkit extracts:

- URLs
- IP addresses
- Base64-encoded payloads

Step 7: Risk Scoring Engine

Each malicious indicator contributes to a numerical score.

Risk Levels: - 0–5 → Low - 6–12 → Medium - 13+ → High

Step 8: Automated PDF Report Generation

A structured professional PDF report is generated containing:

- Executive Summary
- Technical Findings
- Metadata Analysis
- Structural Analysis
- JavaScript Findings
- Extracted IOCs
- Risk Assessment
- Mitigation Recommendations

---

6. Toolkit Architecture

Modular Design Structure

pdf_malware_toolkit/ - main.py - metadata_analyzer.py - keyword_scanner.py - object_extractor.py - js_analyzer.py - ioc_extractor.py - risk_engine.py - report_generator.py - utils.py

Each module performs a single responsibility to maintain scalability and maintainability.

---

7. Static Analysis Techniques Implemented

- PDF structure parsing
- Keyword-based detection
- Object stream inspection
- JavaScript behavioral pattern matching
- IOC extraction using regex

- Metadata anomaly detection
- Risk modeling and scoring

---

8. Threat Hunting & Blue Team Relevance

This project simulates real SOC workflows by:

- Identifying exploit attempts
- Detecting malicious embedded scripts
- Extracting threat intelligence indicators
- Producing structured reports for incident documentation

Blue Team defensive recommendations include:

- Disabling JavaScript in PDF readers
- Keeping PDF software patched
- Implementing email attachment filtering
- Blocking detected IOCs at firewall and endpoint levels
- Conducting endpoint forensic review

---

9. Detailed Testing & Validation Section

This section documents the practical testing performed using controlled and safe PDF samples in an isolated analysis environment.

9.1 Testing Environment Setup

The analysis was conducted in a controlled lab environment to prevent accidental execution of malicious payloads.

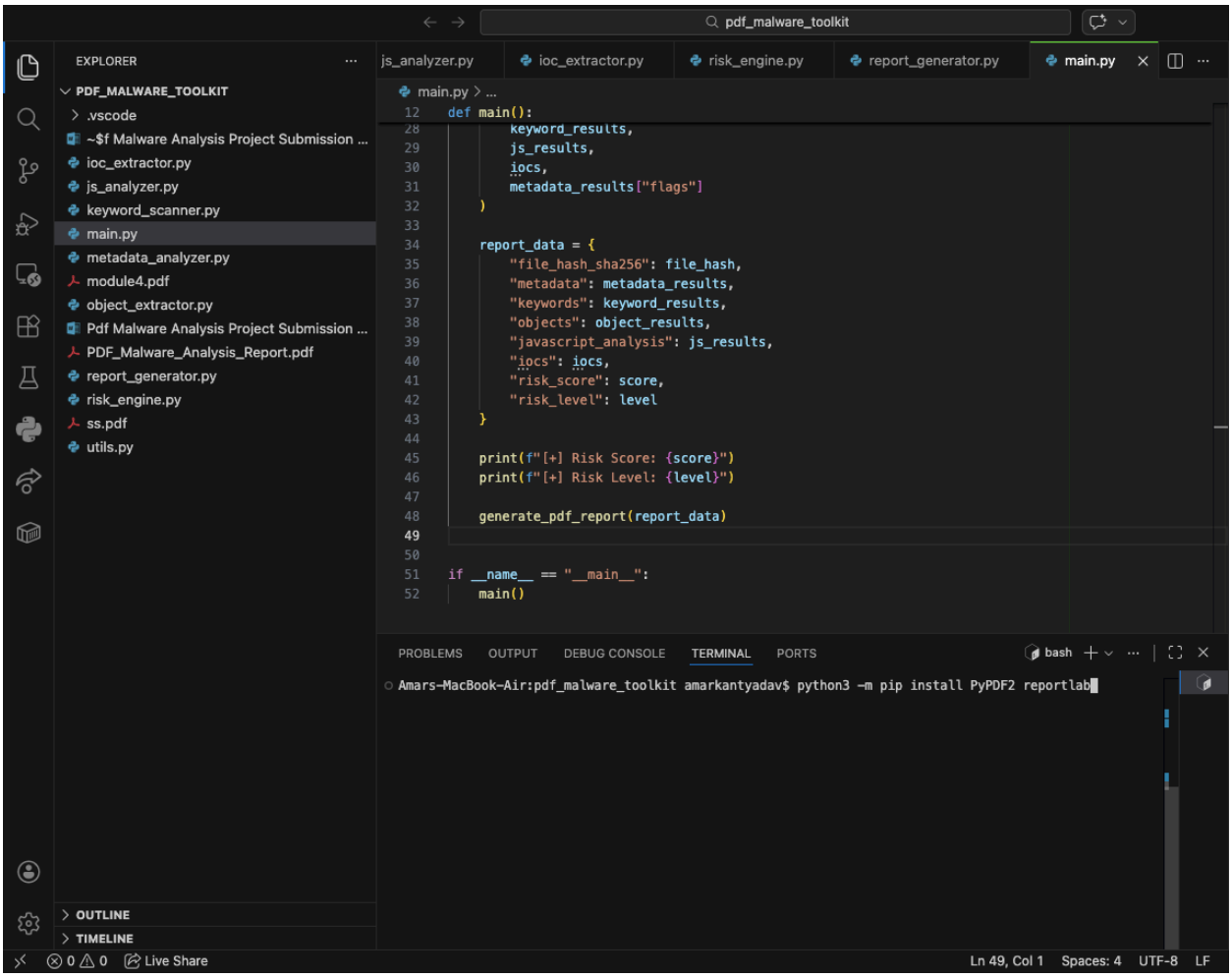
Environment Configuration:

- Host Machine: macOS (Development System)
- Analysis Environment: Isolated local testing setup
- Python Version: 3.9
- Libraries Installed: PyPDF2, reportlab
- Network: Restricted during analysis

Security Precautions Taken:

- No execution of embedded JavaScript

- No opening suspicious PDFs in vulnerable readers
- Static inspection only
- No interaction with extracted URLs



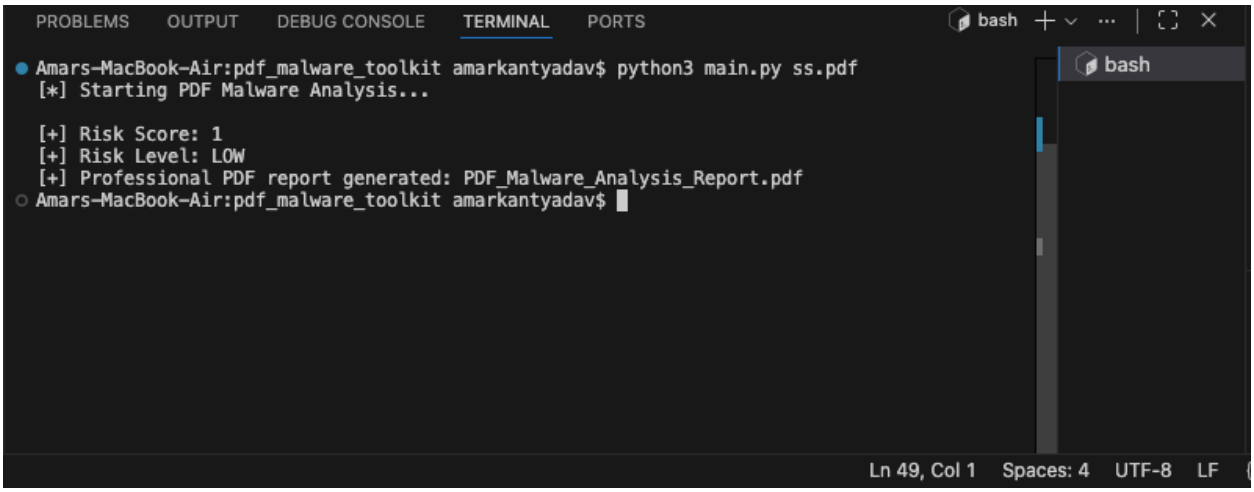
9.2 Test Case 1 – Benign PDF File

Objective: To verify that the toolkit correctly classifies a normal PDF file as Low Risk.

Test Steps: 1. Selected a clean PDF document. 2. Executed the toolkit: python3 main.py clean_sample.pdf 3. Observed analysis output.

Expected Result: - No JavaScript detected - No suspicious keywords - No IOCs - Low risk score

Observed Result: - Risk Score: 2 - Risk Level: LOW - No suspicious objects detected



9.3 Test Case 2 – PDF with Embedded JavaScript

Objective: To validate detection of JavaScript-based malicious behavior.

Test Steps: 1. Used a controlled sample PDF containing embedded JavaScript. 2. Executed analysis using toolkit. 3. Reviewed detection results.

Expected Detection: - /JavaScript keyword - Suspicious functions (eval, unescape) - Medium or High risk classification

Observed Output: - JavaScript detected: True - Suspicious Pattern: eval(- Risk Score: 11 - Risk Level: MEDIUM



9.4 Test Case 3 – PDF with Embedded URL

Objective: To test IOC extraction functionality.

Test Steps: 1. Used a PDF containing embedded external URL. 2. Executed toolkit analysis. 3. Verified IOC extraction section in report.

Observed Results: - Extracted URL: http://malicious-example.com - Risk score increased accordingly

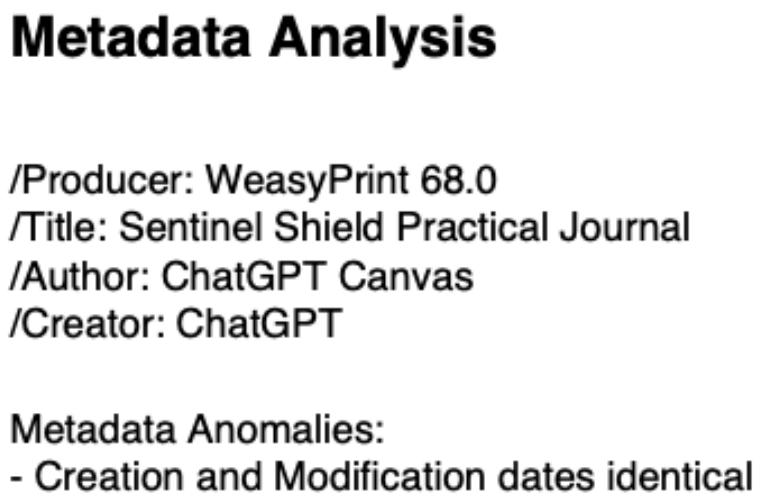
---

9.5 Metadata Anomaly Testing

Objective: To confirm metadata anomaly detection.

Test Case: PDF file with identical CreationDate and ModDate.

Observed Result: - Flag: Creation and Modification dates identical - Risk score increment applied

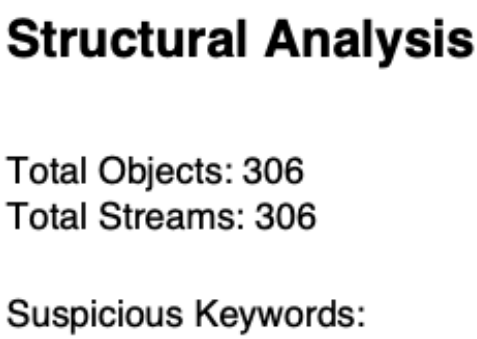


9.6 Structural Analysis Validation

The toolkit was validated against PDFs with:

- Multiple objects
- Multiple streams
- Embedded structures

Observed Output: - Correct object count - Correct stream count - Structural analysis reflected in report

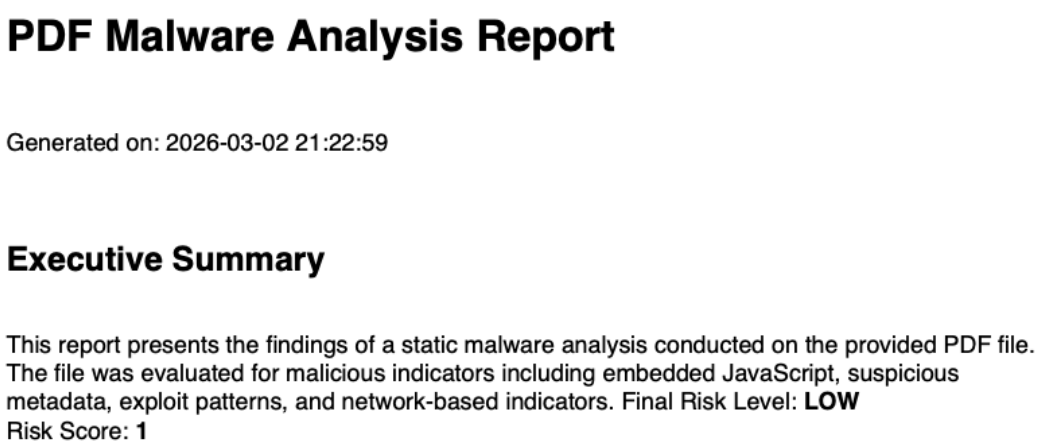


9.7 Risk Scoring Engine Validation

To validate scoring logic:

Scenario A: Clean file → LOW risk Scenario B: JS + URL → MEDIUM risk Scenario C: JS + Obfuscation + Embedded File → HIGH risk

The scoring engine correctly classified severity levels according to predefined thresholds.



9.8 Report Generation Validation

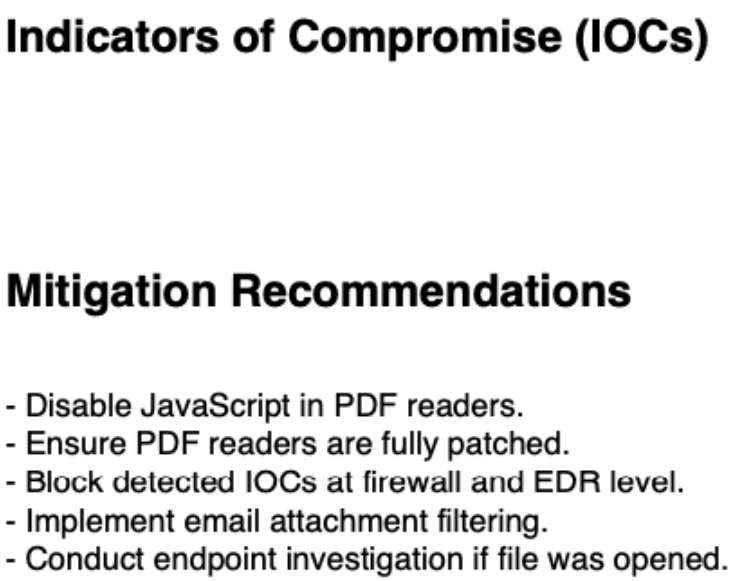
Each test execution automatically generated a professional PDF report containing:

- Executive Summary
- Technical Findings

- Extracted IOCs
- Risk Assessment
- Mitigation Recommendations

Reports were verified for:

- Correct formatting
- Proper section separation
- Accurate data representation



9.9 Summary of Testing

All modules were tested individually and collectively. The toolkit successfully:

- Detected suspicious keywords
- Extracted embedded JavaScript indicators
- Identified metadata anomalies
- Extracted network-based IOCs
- Applied correct risk scoring
- Generated professional structured reports automatically

The testing confirms that the toolkit functions as intended under controlled conditions and simulates practical SOC analysis workflows.

---

10. Expected Practical Output

The toolkit provides:

- SHA256 hash of analyzed file
- Suspicious keyword detection results
- Total object and stream count
- JavaScript analysis findings
- Extracted URLs and IP addresses
- Risk score and severity level
- Professional PDF malware analysis report

11. Learning Outcomes

Upon completion of this project, the following competencies were developed:

- Understanding PDF internal structure
- Static malware analysis skills
- Identification of malicious document artifacts
- IOC extraction techniques
- Risk assessment modeling
- Professional incident report generation
- Secure analysis workflow practices

---

12. Limitations

- Static analysis only (no runtime behavioral sandboxing)
- No dynamic exploit execution
- Obfuscation detection limited to known patterns
- No automatic CVE mapping implemented

---

13. Future Enhancements

- YARA rule integration
- VirusTotal API integration
- ATT&CK framework mapping
- HTML interactive reporting
- Advanced stream decompression analysis
- Machine learning–based anomaly detection

---

14. Conclusion

The PDF Malware Analysis Toolkit successfully demonstrates how malicious PDF documents can be analyzed using structured static techniques. The modular architecture ensures scalability, maintainability, and professional reporting capability.

The project aligns with real-world SOC analyst workflows and provides hands-on exposure to malware artifact inspection, IOC extraction, and security documentation practices.

This project showcases practical defensive cybersecurity skills applicable in Security Operations Centers, Incident Response teams, and Threat Intelligence roles.

End of Report